

Software/ML Domain Taskphase Report

Conceptual report :Taks 1 to 5

Participant name: Samaira Singh

Roll Number: 251090051002

Domain: Software/ML

Repository: https://github.com/samaira-s/Synergy_TP

Submission Date: June 29, 2026

Abstract

This report presents the theoretical background and conceptual understanding developed through the Software/ML Domain Taskphase, covering Tasks 1 to 5. The taskphase involved setting up a version-controlled Python development environment, building CSV parsers both manually and using the pandas library, cleaning a messy real-world dataset, and generating visual plots using matplotlib. Each task was designed to build foundational skills in software engineering and data processing. This report explains the purpose of each tool and method used, the reasoning behind key decisions, and the observations made throughout the process. It does not reproduce code but instead focuses on concepts, design choices, and analytical understanding gained from completing the tasks.

1. Introduction

The Software/ML Domain Taskphase is a structured series of programming assignments designed to build core competencies in software development and data science. The tasks progress from environment setup and version control to data parsing, cleaning, and visualization. Each task introduces a new layer of complexity while reinforcing concepts from the previous one. The goal is not just to write working code but to understand why certain tools and approaches are used, and how raw data transforms into clean, analyzable, and visually communicable information. This report covers the theoretical concepts and design decisions behind Tasks 1 through 5.

2. Development Environment and Version Control

2.1 Git and GitHub

Git is a distributed version control system that tracks changes to files over time. Every change is saved as a commit, which acts as a permanent snapshot of the codebase at a given moment. Each commit carries a unique hash identifier, allowing developers to revisit or compare any previous state of the project. GitHub is a cloud-based platform that hosts Git repositories, enabling collaboration and remote access to code.

Repository hygiene refers to maintaining a clean, organized repository. This includes writing meaningful commit messages that describe what changed and why, keeping unrelated changes in separate commits, and ensuring the repository always contains runnable code. A clean repository communicates professionalism and makes the codebase easier to review and maintain.

2.2 Virtual Environments and requirements.txt

A Python virtual environment is an isolated Python installation created specifically for a project. It ensures that the packages installed for one project do not interfere with those of another. This is important because different projects may depend on different versions of the same library. The virtual environment is created using the built-in venv module and activated before any package installation.

The requirements.txt file records all packages and their exact versions installed in the virtual environment. This file allows any other developer to recreate the same environment using a single command, ensuring reproducibility across different machines and setups.

2.3 .gitignore

The .gitignore file instructs Git to exclude certain files and folders from being tracked. The virtual environment folder is excluded because it contains thousands of auto-generated files that are not part of the project's source code. Compiled Python files and system-generated files such as .DS_Store are also excluded to keep the repository clean and focused on source code only.

3. Python and File Handling Concepts

3.1 Functions and Type Hints

Functions in Python encapsulate reusable blocks of logic. Each function in the taskphase was designed to perform a single, well-defined task, following the principle of separation of concerns. Type hints were added to function signatures to indicate the expected input and output types. While Python does not enforce type hints at runtime, they serve as documentation and help catch errors during development.

3.2 Lists, Dictionaries, and File I/O

Lists and dictionaries are the primary data structures used throughout the tasks. A list holds an ordered collection of items, while a dictionary maps keys to values. CSV rows were represented as dictionaries where each column name is a key and the corresponding cell value is the value. File I/O was handled using Python's built-in open() function with context managers, which ensure files are properly closed after use even if an error occurs.

3.3 Exception Handling

Exception handling using try and except blocks was used to manage errors gracefully. Rather than allowing the program to crash with an unhelpful error, specific exceptions such as

`FileNotFoundError` and `ValueError` were raised with clear messages. This makes the code more robust and easier to debug when something goes wrong.

4. CSV Parsing and Pandas

4.1 CSV as a Data Format

CSV stands for Comma-Separated Values. It is a plain text format where each line represents a row of data and values within each row are separated by commas. The first line is typically a header row that names each column. CSV is widely used because it is simple, human-readable, and supported by almost every data tool. However, it has limitations such as no built-in support for data types, making type conversion a necessary step after loading.

4.2 Manual CSV Parsing

In Task 3, the CSV file was parsed manually using only Python's built-in file reading capabilities. The file was opened and read line by line. The header line was split by comma to extract column names. Each subsequent line was similarly split and matched to the header to construct a dictionary representing one row. Empty lines and rows with the wrong number of values were skipped to handle malformed data without crashing. This approach demonstrates the underlying mechanics that libraries like pandas abstract away.

4.3 Pandas-Based Loading

Pandas is a Python library for data manipulation and analysis. Its `read_csv()` function loads a CSV file into a `DataFrame`, which is a two-dimensional table with labeled rows and columns. Pandas automatically infers column names from the header row and provides a wide range of built-in operations for filtering, grouping, aggregating, and transforming data. The same summary statistics calculated manually in Task 3 were reproduced using pandas to confirm that both approaches produce identical results.

5. Data Cleaning

5.1 Why Cleaning is Necessary

Real-world datasets are rarely clean. They often contain missing values, duplicate rows, inconsistent formatting, incorrect data types, and invalid values. Feeding dirty data into any analysis or machine learning model produces unreliable results. Data cleaning is the process of identifying and correcting these issues systematically before any further analysis is performed.

5.2 Cleaning Steps Applied in Task 4

The messy dataset used in Task 4 contained a range of common data quality issues. Duplicate rows were identified and removed using pandas. Domain names appeared in multiple inconsistent formats and were standardized to four canonical values using a mapping dictionary. Attendance values contained percentage symbols and invalid entries below 0 or above 100, which were removed and replaced with the column median. Score and study hours columns contained word-form numbers such as nine and two, which were converted to integers using a lookup dictionary. Height values were given in both centimetres and metres and were unified into centimetres. Weight values had inconsistent units and spacing around the kg suffix, which were stripped and converted to floats. Submitted values appeared as yes, Yes, Y, no, No, and N, all of which were normalized to lowercase yes or no.

5.3 Missing Value Strategy

Missing values were handled column by column with a documented rule for each. Numeric columns with missing values were filled with the column median rather than the mean, because the median is less sensitive to outliers. The submitted column was filled with no as a conservative default, assuming that a missing submission record means the student did not submit. Each decision was recorded in the cleaning report rather than silently applying a blanket rule.

5.4 Validation After Cleaning

After cleaning, a separate validation function was run to verify that the data met all expected constraints. Checks included confirming that student IDs were unique, attendance values fell within the 0 to 100 range, all numeric columns contained only numeric data, submitted contained only yes or no, domain contained only the four valid values, and no critical columns contained missing values. Validation provides confidence that the cleaning steps worked correctly and that the output data is safe to use for further analysis.

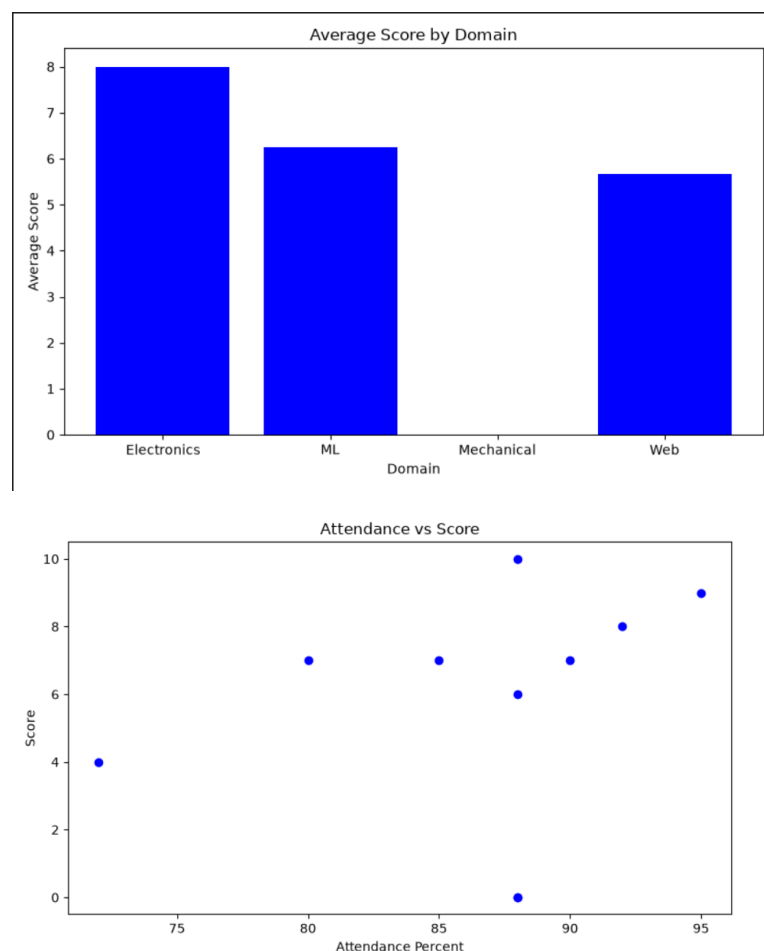
6. Data Visualization

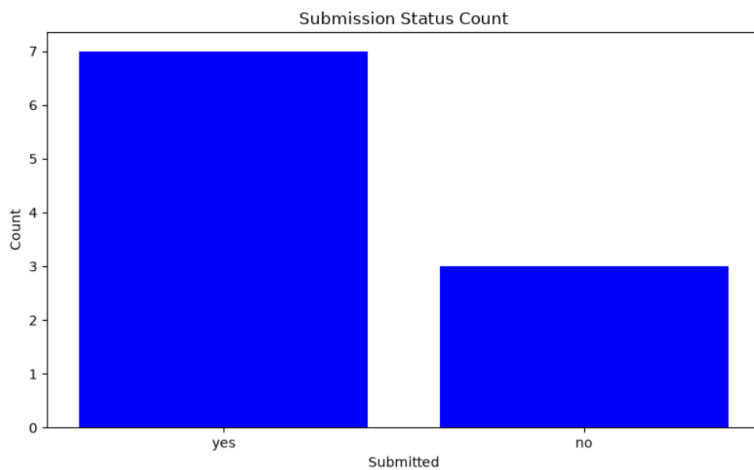
6.1 Purpose of Visualization

Data visualization translates numerical data into visual representations that are easier for humans to interpret. Patterns, trends, and anomalies that are difficult to spot in a table of numbers become immediately apparent in a well-designed chart. In Task 5, three plots were generated from the cleaned dataset to communicate key findings visually.

6.2 Plot Descriptions

The first plot is a bar chart showing the average score for each domain. This makes it easy to compare performance across ML, Web, Electronics, and Mechanical domains at a glance. The second plot is a scatter plot showing the relationship between attendance percentage and score. Each point represents one student, and the overall pattern suggests whether higher attendance correlates with higher scores. The third plot is a bar chart showing the count of students who submitted versus those who did not, providing a quick summary of submission rates.





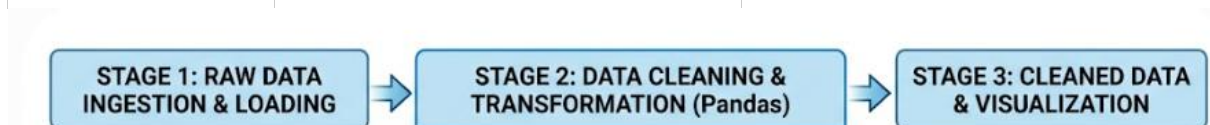
6.3 Plot Requirements

Each plot was given a clear title, labeled x-axis and y-axis, and was saved to disk using matplotlib's `savefig()` function. The `tight_layout()` function was applied to prevent axis labels from being cut off. The non-interactive Agg backend was used so that plots could be generated and saved in an environment without a display, which is important for scripts run from the command line or on remote servers.

7. Task Summary

The following table summarizes each task and the main concept learned from it.

Task	Description	Main Concept Learned
Task 1	GitHub setup, virtual environment, Linux commands	Version control and environment isolation
Task 2	Python CSV analyzer with type hints and error handling	Functions, file I/O, and exception handling
Task 3	Manual CSV parser vs pandas comparison	How CSV parsing works at a low level
Task 4	Messy dataset cleaning and validation	Systematic data cleaning and validation
Task 5	Matplotlib visualizations from cleaned data	Translating data into visual insights



8. Conclusion

The Software/ML Domain Taskphase provided a structured progression through the foundational skills required for software development and data science work. Starting from setting up a clean development environment and learning version control, the tasks moved through data parsing, cleaning, and visualization. Each step built on the previous one, demonstrating how raw data passes through multiple stages of processing before it becomes useful for analysis or machine learning.

The manual CSV parser in Task 3 provided insight into what higher-level libraries like pandas do automatically. The data cleaning task in Task 4 highlighted how much preparation real-world data requires before it can be trusted. The visualization task in Task 5 demonstrated how cleaned data can be communicated visually to reveal patterns that are not obvious from the raw numbers. Together, these tasks form a complete data processing pipeline from raw input to visual output.

References

1. Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
2. pandas Development Team. pandas Documentation. <https://pandas.pydata.org/docs/>
3. Hunter, J. D. Matplotlib: A 2D Graphics Environment. <https://matplotlib.org/>
4. Git Documentation. <https://git-scm.com/doc>
5. GitHub Documentation. <https://docs.github.com/>